

SISTEMA DE SIMULACIÓN DE COMBATE E INVENTARIO PARA UN VIDEOJUEGO RPG

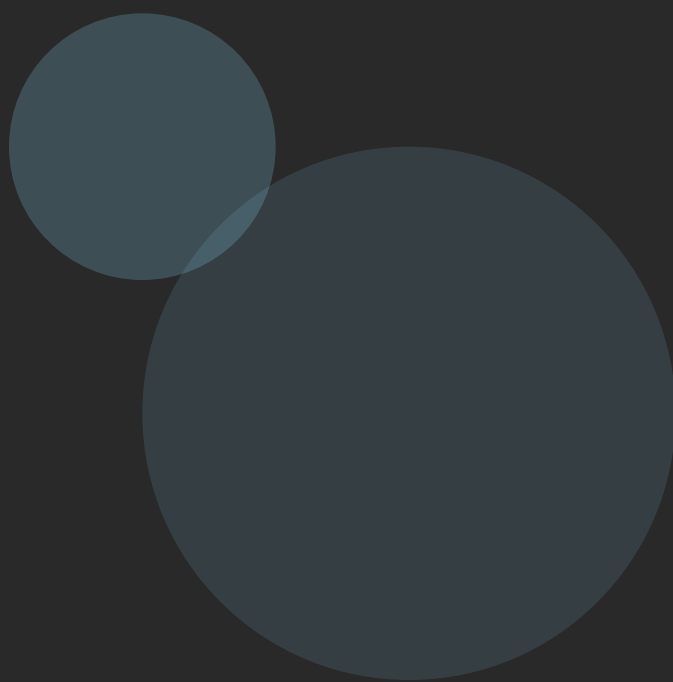


Patrones de Diseño de Software

Binciguerra, Santiago | Gomes Opitz, Mariano

Licenciatura en Informática

Universidad Nacional de Tucumán | 2026



Objetivo del Sistema y Módulo A



OBJETIVO GENERAL

Diseñar e implementar el núcleo lógico (backend) para un sistema de videojuegos de rol (RPG) basado en turnos. El sistema deberá gestionar de forma robusta la creación de personajes, la alteración de sus comportamientos durante el combate, la organización jerárquica de sus pertenencias, la modificación dinámica de su equipamiento y la notificación de eventos hacia sistemas secundarios del juego.

MODULO A: GESTIÓN Y CREACIÓN DE COMBATIENTES

- Cuatro clases seleccionables: **Luchador, Barbaro, Arquero, Hechicero**
- Atributos diferenciados: Fuerza, Percepcion, Resistencia, Carisma, Inteligencia, Agilidad, Suerte, PV
- Inicialización centralizada y desacoplada del flujo de combate
- Extensible para nuevas clases: Clerigos, Bardos, Caballeros

Modulos B y C: Combate e Inventario



MODULO B: MECÁNICA DE COMBATE Y ESTADOS ALTERADOS

Los personajes realizan acciones (Atacar, Defender, Usar Habilidad) cuya efectividad depende del estado actual:

Saludable - Acciones normales con plena efectividad

Envenenado - Pierde % de PV al inicio de cada turno

Paralizado - Pierde el turno completamente

Muerto - No recibe ataques ni toma turnos

MODULO C: SISTEMA DE INVENTARIO JERÁRQUICO

Cada personaje posee un almacenamiento que maneja:

Objetos simples - Pociones, Gemas, Armas (hojas)

Contenedores - Bolsas, Cofres (compuestos)

Los contenedores albergan objetos u otros sub-contenedores de forma indefinida.

Calcula **Peso Total** y **Valor en Oro** de forma recursiva y transparente.

Modulos D y E: Equipamiento y Eventos



MODULO D: MODIFICADORES DINÁMICOS DE EQUIPAMIENTO

Las armas base poseen dano fijo, pero permiten mejoras dinámicas:

Modificador de Fuego - Añade un bonus daño

Modificador de Filo - Multiplica el daño

Modificador de Veneno - Añade un pequeño bonus de daño, y permite envenenar o paralizar al enemigo.

Las mejoras se **apilan infinitamente** en tiempo de ejecución.

Altera **CalcularDano()** sin crear clases rígidas para cada combinación.

MODULO E: DESACOPLAMIENTO DE EVENTOS GLOBALES

Acciones críticas tienen repercusiones en sistemas secundarios:

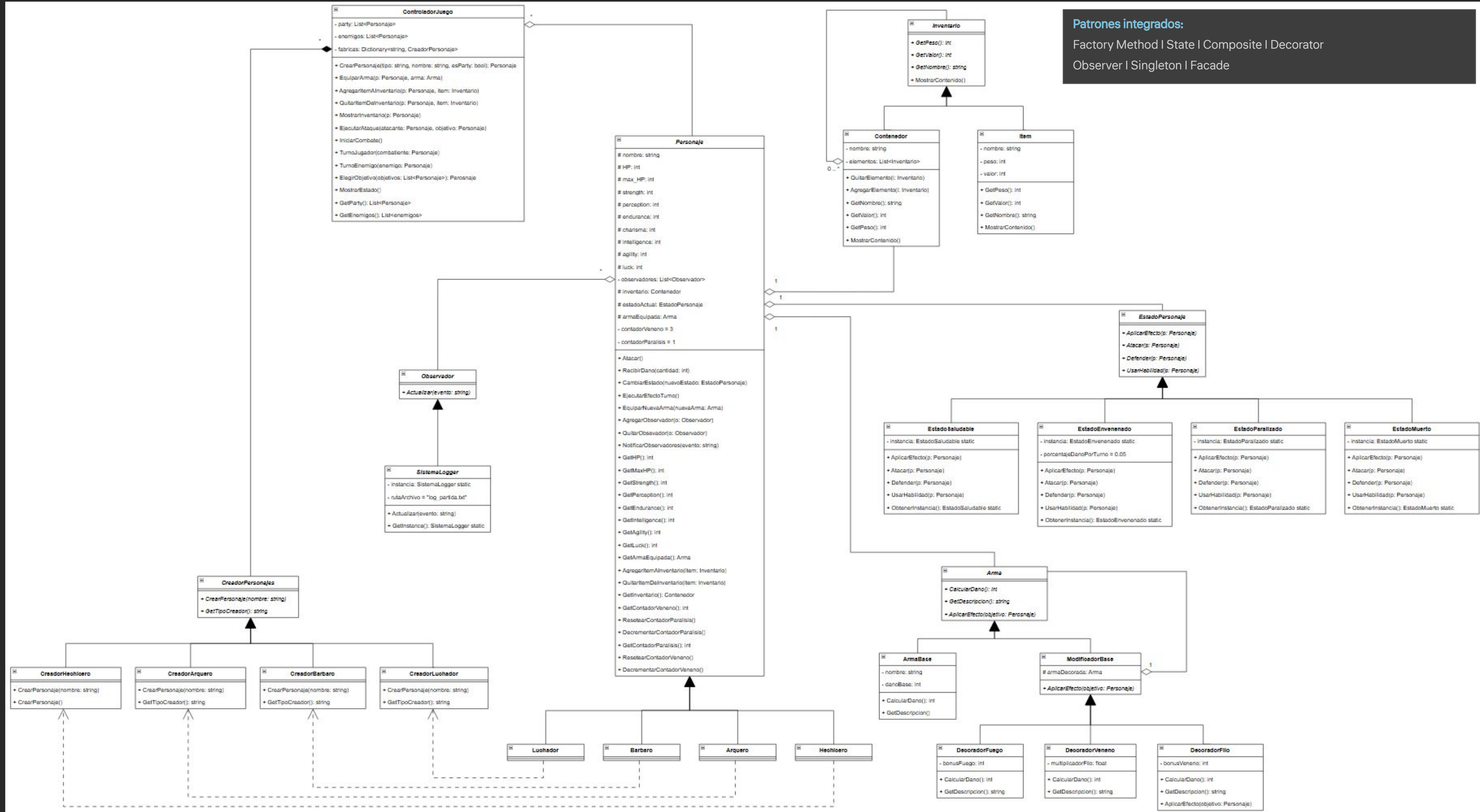
Sistema de Registro (Logger) - Asienta detalles de decesos en archivo de texto

La clase Personaje **no conoce** la existencia de estos sistemas.

Solo emite un **aviso general** cuando cambian sus estadísticas.

Permite que la cantidad de sistemas aumente o disminuya dinámicamente.

Diagrama Principal del Sistema



Factory Method - Creación de Objetos



PROPOSITO

Crear objetos sin especificar la clase exacta del objeto que se creara.

BENEFICIO CLAVE

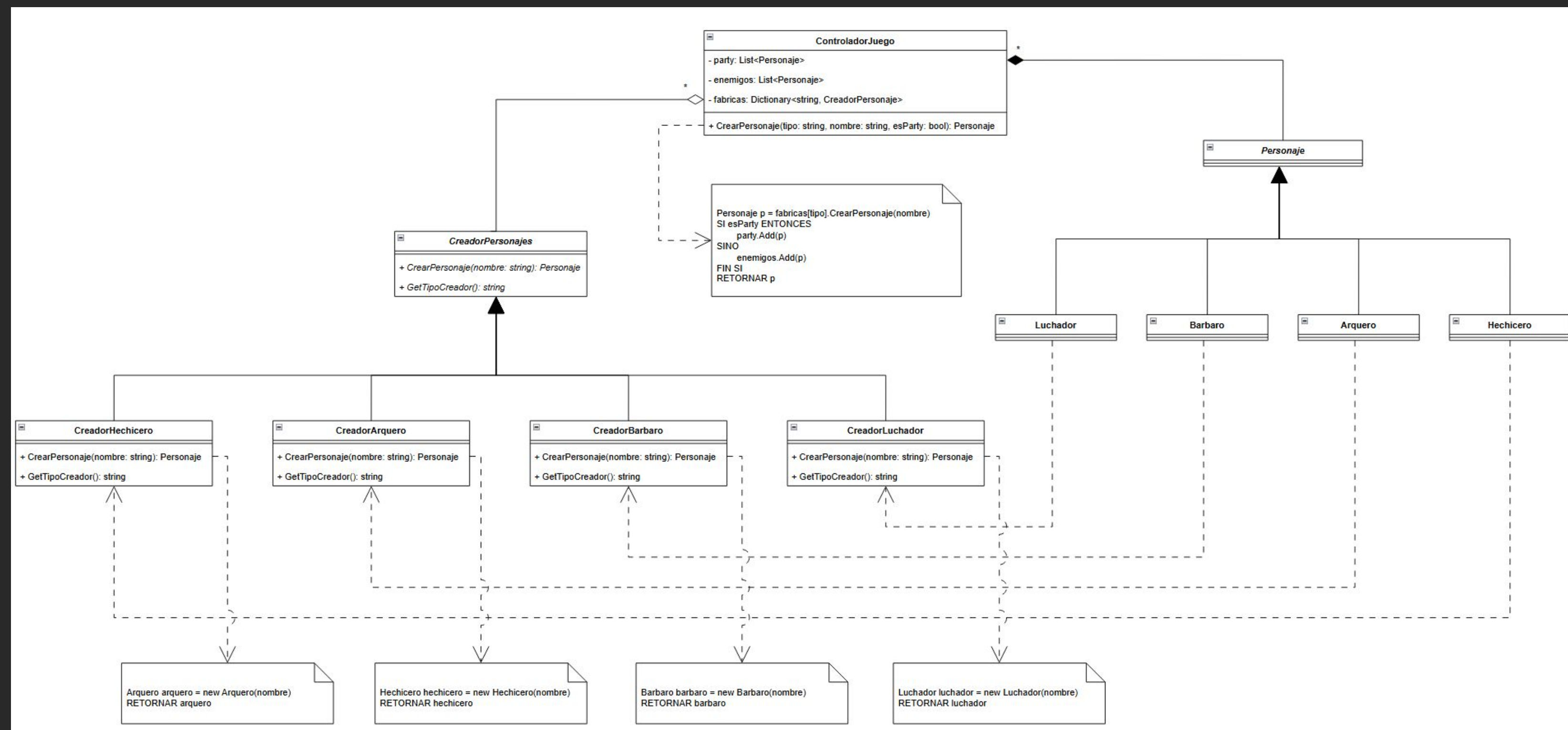
Permite incorporar nuevas clases (Clerigos, Bardos, Caballeros) **sin modificar** el código que gestiona la partida.

APLICACIÓN EN EL SISTEMA

Se define una interfaz **CreadorPersonajes** con método abstracto **CrearPersonaje()**. Las subclases concretas (**CreadorGuerrero**, **CreadorMago**) implementan este método para instanciar y parametrizar cada tipo de combatiente.

Esto permite que el bucle del juego solicite personajes de forma **genérica** a través de la abstracción, encapsulando las reglas de inicialización.

DIAGRAMA DE CLASES



State - Comportamiento Dependiente del Estado



PROPOSITO

Alterar el comportamiento de un objeto cuando su estado interno cambia.

APLICACIÓN EN EL SISTEMA

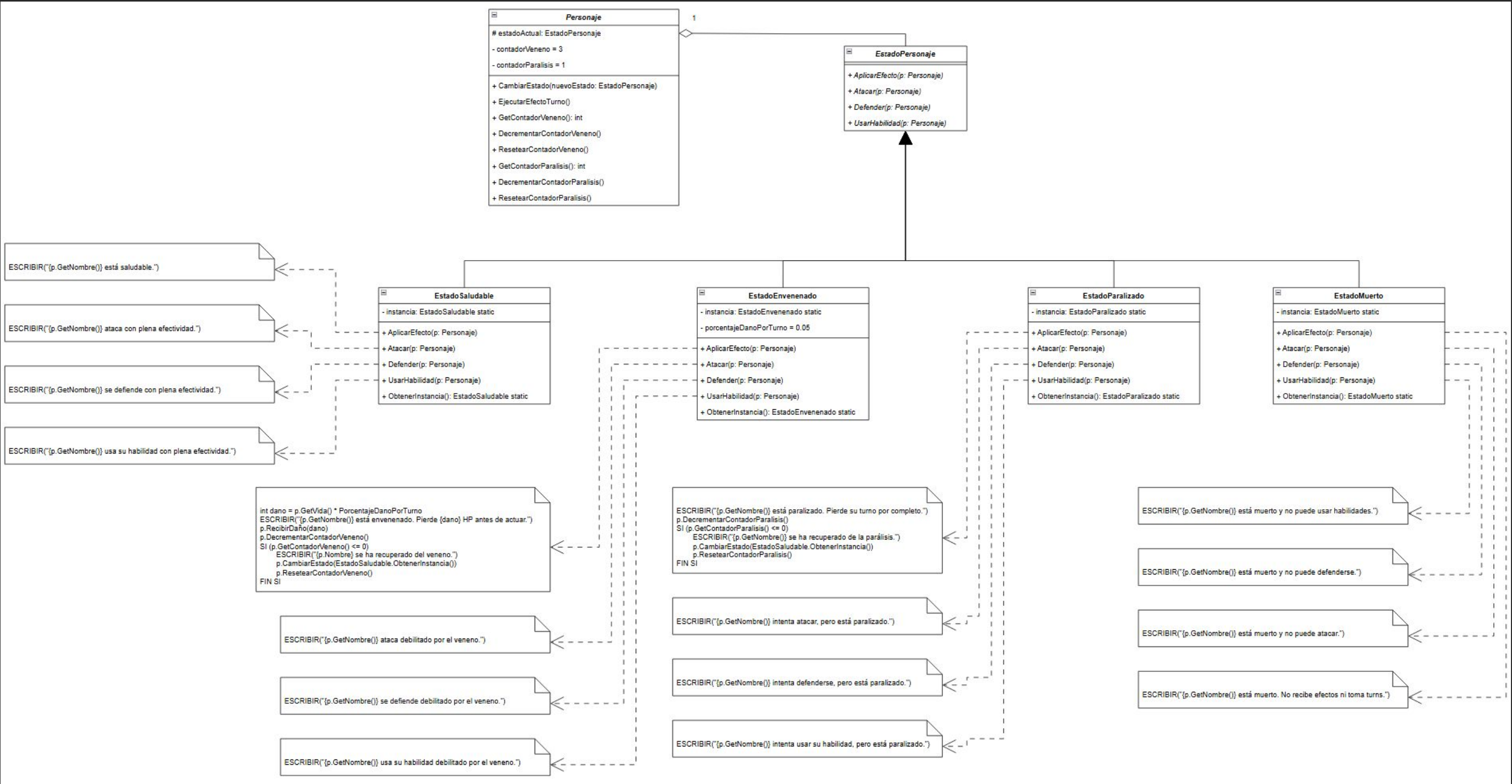
Se extrae el comportamiento en una jerarquia **EstadoPersonaje** con clases concretas:

EstadoSaludable, EstadoEnvenenado, EstadoParalizado, EstadoMuerto .

El personaje delega la ejecución al estado actual (**estadoActual.Atacar()**), eliminando condicionales anidados y permitiendo transiciones limpias entre estados.

- Saludable** - Acciones con plena efectividad
- Envenenado** - Pierde PV automaticamente cada turno
- Paralizado** - Pierde el turno, salta al siguiente
- Muerto** - No actúa ni recibe modificaciones

DIAGRAMA DE CLASES



Composite - Estructuras Jerárquicas Parte-Todo



PROPOSITO

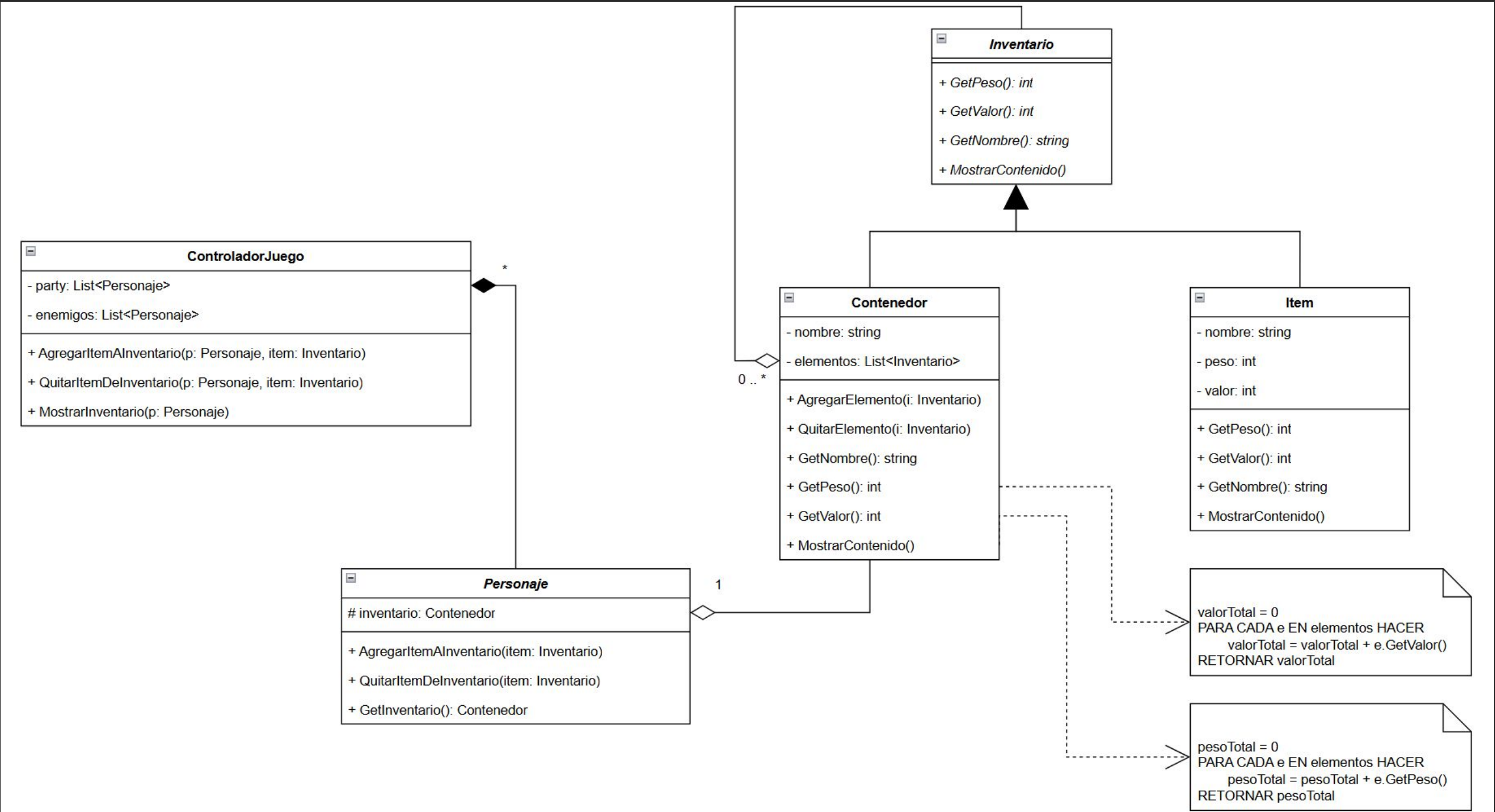
Componer objetos en estructuras de árbol para representar jerarquías parte-todo.

APLICACIÓN EN EL SISTEMA

Se establece la interfaz **Inventario** con operaciones **GetPeso()** y **GetValor()** .
Los objetos simples (**Item**) actúan como hojas; los contenedores (**Contenedor**) actúan como compuestos con lista interna de tipo Inventario.
Al invocar **GetPeso()** en un contenedor, este recorre su lista sumando recursivamente los pesos, tratando elementos simples y compuestos bajo una **misma interfaz unificada** .

El cliente trabaja con objetos individuales y composiciones de forma **uniforme** , sin distinguir entre hojas y compuestos.

DIAGRAMA DE CLASES



Decorator - Añadir Responsabilidades Dinámicamente



PROPOSITO

Añadir funcionalidades a objetos de forma dinámica sin alterar su estructura.

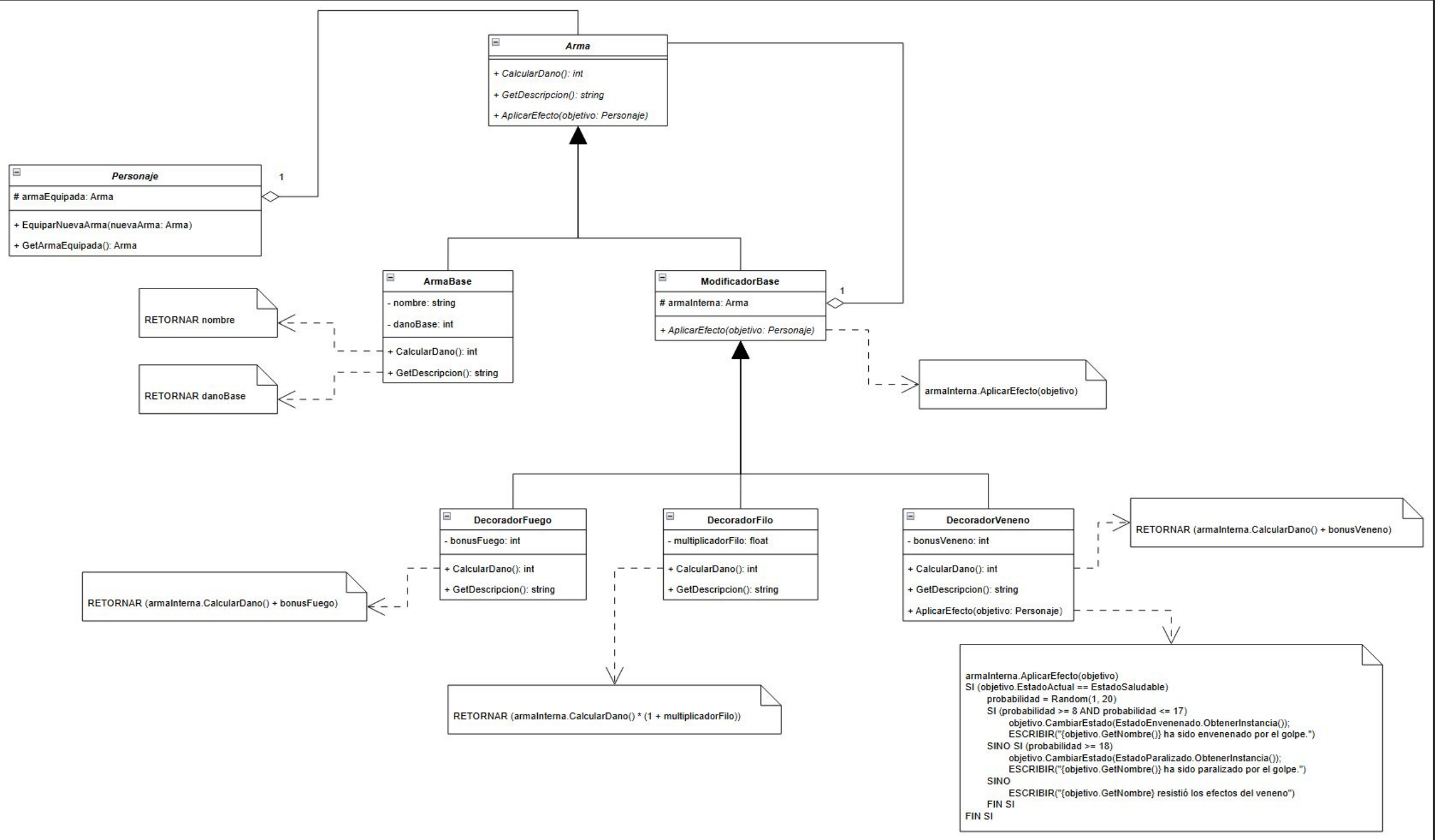
APLICACIÓN EN EL SISTEMA

Se define **ModificadorArma** que implementa la interfaz **Arma** y contiene una referencia agregada a otro objeto Arma. Los decoradores concretos (**DecoratorFuego**, **DecoratorFilo**, **DecoratorVeneno**) heredan del modificador base.

Al sobrescribir **CalcularDano()** , cada decorador ejecuta el cálculo del arma interna y añade su propia bonificación, permitiendo **envolturas en tiempo de ejecución** .

```
new DecoradorFuego(new DecoradorFilo(new Arma()))
```

DIAGRAMA DE CLASES



Observer - Notificación de Eventos

PROPOSITO

Definir una dependencia uno-a-muchos entre objetos para notificación automática.

APLICACIÓN EN EL SISTEMA

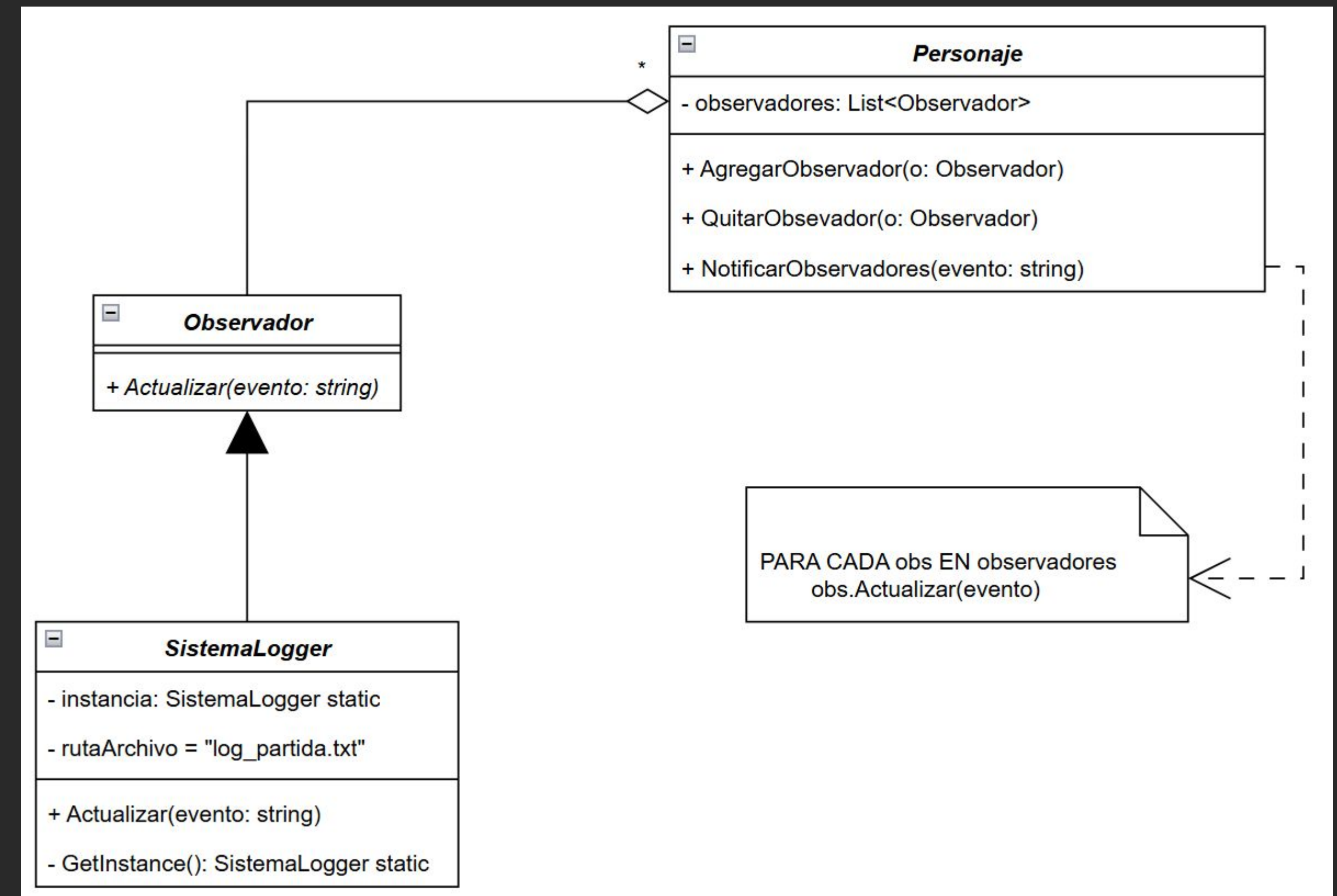
La clase **Personaje** asume el rol de **Sujeto Concreto** , manteniendo una lista de suscriptores a través de la interfaz **Observador** . El componente periférico (**SistemaLogger**) implementa dicha interfaz.

Cuando la vida del personaje se altera o llega a cero, invoca automáticamente **NotificarObservadores()** , provocando que los sistemas adheridos ejecuten su lógica de respuesta de forma **totalmente desacoplada** .

La clase Personaje **no conoce** la existencia de los sistemas secundarios; solo emite avisos generales.



DIAGRAMA DE CLASES



Singleton - Instancia Unica Global



PROPOSITO

Garantizar que una clase tenga una única instancia y proporcionar un acceso global a ella.

APLICACIÓN EN EL

SISTEMA

1. Estados Concretos (Integracion con State)

Las clases de estados no poseen variables de instancia propias; encapsulan algoritmos de comportamiento que operan sobre el personaje recibido por parámetro.

Como Singletons con constructor privado, se evita la creación masiva de objetos idénticos. Todos los personajes **comparten la misma instancia** de cada estado.

2. Sistemas Observadores (Integracion con Observer)

Los observadores manejan recursos únicos. El **SistemaLogger** administra acceso exclusivo a un archivo de texto en disco.

Como Singletons, se garantiza que no haya múltiples instancias escribiendo en el mismo archivo simultáneamente, evitando **bloqueos por concurrencia** y asegurando un punto de acceso seguro.

Beneficio clave: Reduce la fragmentación de memoria y optimiza el rendimiento del bucle principal del juego, proveyendo un **punto de acceso seguro** desde cualquier sección del motor lógico.

Facade - Interfaz Unificada del Sistema



PROPOSITO

Proveer una interfaz unificada que simplifica el uso de un conjunto de interfaces.

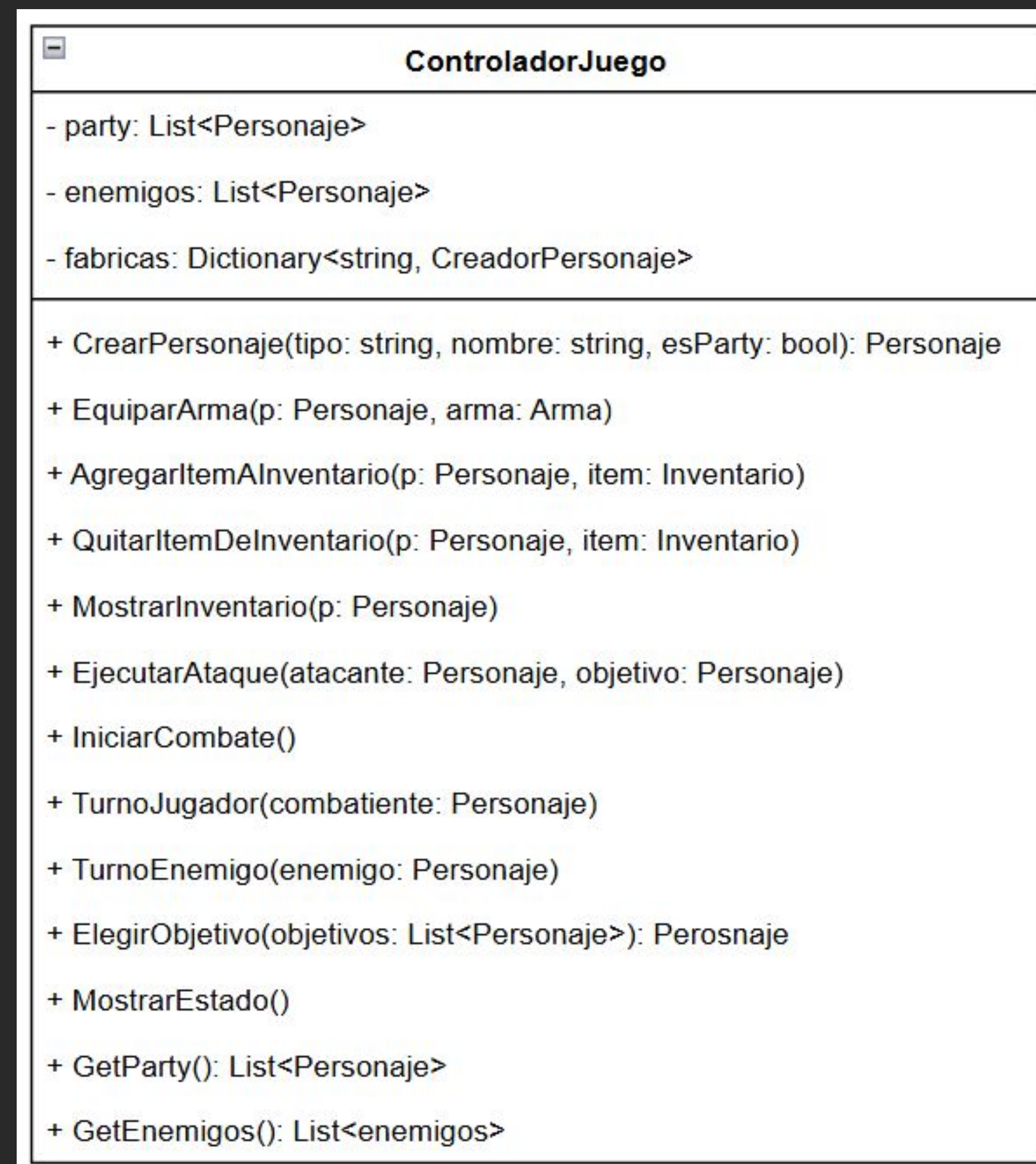
APLICACIÓN EN EL SISTEMA

Previne exponer directamente la lógica del sistema (creadores, estados, decoradores) a la interfaz de usuario. La clase **ControladorJuego** actúa como el **director del sistema**.

Tiene la responsabilidad de instanciar componentes iniciales, coordinar acciones y mantener el **estado global** de la partida: gestiona fabricas, entidades, observadores, inicia partidas, ejecuta ataques y consulta inventarios.

Evita **alto acoplamiento** entre la UI y la lógica interna, haciendo el sistema más fácil de mantener.

DIAGRAMA DE CLASES



Gracias

Patrones de Diseño en un Sistema RPG

Binciguerra, Santiago | **Gomes Opitz, Mariano**

Licenciatura en Informática | Facultad de Ciencias Exactas y Tecnología

Departamento Ciencias de la Computación

Universidad Nacional de Tucuman | 2026

github.com/Divitrion/Proyecto-Patrones

Fuente: Refactoring Guru - Design Patterns Catalog

